MAJOR PROJECT REPORT

Automated and Scalable Platform for Subscription Management with Alerts (ES-452)

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

Computer Science & Engineering

Supervisor:
Ms. Apurva Jain
Assistant Professor

Submitted by:
Akshat Jain
00115607223

Department of Computer Science and Engineering

Dr. Akhilesh Das Gupta Institute of Professional Studies

(Formerly ADGITM)

FC-26, SHASTRI PARK, NEW DELHI

Affiliated to

GURU GOBIND SINGH INDRAPRASTHA UNIVERSITY

Sector - 16C Dwarka, Delhi - 110075, India

2022-26

DECLARATION

This is to certify that the material embodied in this Major Project - Dissertation titled "**Automated and Scalable Platform for Subscription Management with Alerts**" being submitted in the partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science & Engineering is based on my original work. It is further certified that this Major Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. My indebtedness to other works has been duly acknowledged at the relevant places.

Akshat Jain
00115607223

CERTIFICATE

This is to certify that the work embodied in this Major Project - Dissertation titled **"Automated and Scalable Platform for Subscription Management with Alerts"** being submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science & Engineering**, is original and has been carried out by **Akshat Jain (00115607223)** under my supervision and guidance.

It is further certified that this Major Project - Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

Ms. Apurva Jain
Assistant Professor

Dr. Megha Gupta
HOD, CSE
Dr. Akhilesh Das Gupta Institute of Professional Studies

Acknowledgement

I would like to express my sincere gratitude to my project guide for their continuous guidance, valuable suggestions, and encouragement throughout the development of this project. Their support played a crucial role in shaping the direction and successful completion of this work.

I am also thankful to the faculty members of the Computer Science and Engineering department for providing the necessary knowledge, resources, and academic support during the course of this project.

I would like to extend my appreciation to my institution for providing a conducive environment and the required infrastructure to carry out this work effectively.
Finally, I would like to thank my family and friends for their constant support, motivation, and encouragement, which helped me stay focused and complete this project successfully.

Akshat Jain
00115607223

Abstract

Subscription-based services have become an integral part of modern digital lifestyles, ranging from streaming platforms and cloud storage to software tools and online memberships. However, managing multiple subscriptions manually often leads to missed renewal dates, unnecessary expenses, and a lack of visibility into recurring financial commitments.

The project **"Automated and Scalable Platform for Subscription Management with Alerts (Sub Track AI)"** presents a centralized and intelligent system designed to help users efficiently track, manage, and analyse their subscriptions. The platform enables users to monitor billing cycles, calculate recurring expenses, and receive timely alerts before subscription renewals.

A key feature of the system is the integration of an AI-powered event generation module using a Large Language Model (LLM). This module allows users to input subscription details in natural language, such as "I subscribed to Netflix for ₹499 per month," which are automatically processed to extract relevant information including subscription name, cost, billing cycle, and renewal date. The system then generates structured subscription events and stores them in the database, significantly reducing manual effort and enhancing user experience.

The platform is built using a microservices-based architecture with Java Spring Boot for backend services, JWT-based authentication for security, and REST APIs for communication. The mobile application is developed using React Native, providing a user-friendly interface for managing subscriptions and viewing analytics. Docker is used for containerization to ensure scalable and consistent deployment.

The system aims to improve financial awareness, minimize unnecessary recurring expenses, and demonstrate the practical implementation of modern software engineering practices combined with AI-driven automation.

TABLE OF CONTENTS

Page No

Declaration……..I
Certificate……...……....II
Acknowledgement…….……...……III
Abstract……...…….....IV

List of Figures
List of Tables

Chapter 1: Introduction……..1
Chapter 2: Problem Statement……..……..6
2.1: Problem Definition……..6
2.2: Objectives……..……7
Chapter 3: Analysis……...……9
3.1: Software Requirement Specifications……9
3.1.1: Functional Requirements of the Project…….……10
3.1.2: Non-functional Requirements of the Project….……12
3.2: Feasibility Study of the Project…….…13
3.3: Tools / Technologies / Platform used……14
3.4: Architecture Diagrams…….……15
Chapter 4: Design and Architecture…….……16
4.1: Structure Chart / Work Breakdown Structure……16
4.2: Explanation of Modules……...….16
4.3: Flow Chart / Activity Diagram……18
4.4: ER Diagram / Class Diagram……...……19
Chapter 5: Implementation……...…….20
5.1: Screenshots……...…….20
5.2: Source Code of some modules…….……27
Chapter 6: Testing…….31
Chapter 7: Summary and Conclusion……...…….34
Chapter 8: Limitation of the Project and Future Work…...……36
Bibliography…….39

LIST OF FIGURES

Figure No.	Figure Title	Page No.
3.4	Architecture Diagram	15
4.3.1	Flow Chart	18
4.3.2	Activity Diagram	19
4.4	ER Diagram	19
5.1	User Registration	20
5.2	Dashboard	21
5.3	Subscription	22
5.4	Add Subscription	22
5.5	Subscription Details	23
5.6	Analytic Page	23
5.7	Notification	24
5.8	Help/Support Page	24
5.9	Profile	25
5.10	Onboarding Main	26
5.11	Reminder Main	26

LIST OF TABLES

Table No.	Table Title	Page No.
6.3	Test Case	32

CHAPTER 1: Introduction

1.1 Background of the Problem

In today's rapidly evolving digital ecosystem, subscription-based services have emerged as one of the most dominant business models across a wide range of industries, including entertainment, software, cloud computing, education, healthcare, and e-commerce. Platforms offering streaming content, Software-as-a-Service (SaaS) applications, cloud storage, online learning platforms, and premium memberships increasingly rely on recurring billing mechanisms to generate consistent and predictable revenue streams.

This shift from one-time purchases to subscription-based models has provided significant advantages to both service providers and users. For businesses, it ensures steady cash flow and customer retention, while for users, it offers flexibility, affordability, and access to continuously updated services. However, as the number of subscription services continues to grow, users often find themselves subscribed to multiple platforms simultaneously, leading to increased complexity in managing these subscriptions effectively.

One of the primary challenges faced by users is the lack of visibility over their active subscriptions. Many individuals subscribe to services on a trial basis or for short-term use but forget to cancel them before the billing cycle renews. This results in unintended charges, financial inefficiencies, and, in some cases, significant cumulative expenses over time. Studies and surveys have indicated that a large percentage of users underestimate their monthly subscription spending due to poor tracking and fragmented information.

In addition to individual users, small organizations and startups also encounter difficulties in managing multiple software subscriptions used for productivity, collaboration, and infrastructure. Without a centralized system, tracking renewal dates, usage patterns, and associated costs becomes a tedious and error-prone task.

Traditional approaches to subscription tracking, such as maintaining manual records in spreadsheets or setting reminders in calendars, are increasingly inadequate in addressing these challenges. These methods lack automation, scalability, and real-time synchronization with dynamic subscription data. Furthermore, they do not provide

intelligent insights, categorization, or integration with external systems such as payment gateways or notification services.

Another critical issue is the absence of proactive alert systems. Most users rely on memory or occasional checks to manage subscriptions, which is unreliable. Without timely reminders, users are often unaware of upcoming renewals, leading to missed opportunities to cancel or modify subscriptions based on their current needs.

As the digital landscape continues to expand, the need for an intelligent, automated, and scalable subscription management solution becomes increasingly evident. Such a system should not only track subscriptions but also provide timely alerts, analytics, and a seamless user experience to ensure effective financial and operational control.

1.2 Motivation

The motivation behind the development of an automated and scalable subscription management platform stems from both practical and technical considerations. On the practical side, the growing dependency on digital services has made subscription management a real-world problem affecting millions of users globally. The inconvenience and financial impact caused by unmanaged subscriptions highlight the necessity for a dedicated solution.

Many users experience "subscription fatigue," where the sheer number of active subscriptions becomes overwhelming. This often leads to negligence in monitoring expenses and subscription statuses. The lack of awareness regarding renewal dates and billing cycles results in avoidable financial losses. Addressing this issue through automation and intelligent alerting can significantly improve user experience and financial discipline.

From a technical standpoint, this project presents an opportunity to design and implement a system that incorporates modern software engineering principles and scalable architecture. Building such a platform involves tackling challenges such as handling concurrent users, managing large datasets, ensuring data consistency, and delivering real-time notifications. These challenges make the project both intellectually stimulating and practically valuable.

The project also enables the exploration of advanced backend technologies and architectural patterns, such as microservices architecture, event-driven systems, and

distributed caching. These technologies are widely used in industry-scale applications and provide valuable hands-on experience in building high-performance systems.

Another important motivation is to bridge the gap between theoretical knowledge and practical implementation. Concepts learned in subjects like database management systems, operating systems, and software engineering can be applied to develop a real-world solution. This not only reinforces academic learning but also enhances problem-solving skills and technical proficiency.

Additionally, the project aims to create a user-centric system that prioritizes usability and efficiency. By integrating features such as automated alerts, categorized dashboards, and intuitive data representation, the platform seeks to empower users with better control over their subscriptions. This aligns with the broader goal of leveraging technology to simplify everyday tasks and improve quality of life.

1.3 Scope of the Project

The scope of this project encompasses the design, development, and implementation of a comprehensive platform for managing subscriptions with automated alert functionality. The system is intended to cater primarily to individual users and small-scale organizations, providing them with a centralized solution to track and manage their subscription-related activities.

The platform focuses on delivering core functionalities that address the fundamental challenges associated with subscription management. These functionalities include the ability to add, update, and delete subscription records, enabling users to maintain an up-to-date database of all their active services. Each subscription entry includes essential details such as service name, billing cycle, cost, start date, and renewal date.

Another key aspect of the system is the tracking of billing cycles and renewal schedules. The platform continuously monitors subscription timelines and identifies upcoming renewals. Based on predefined conditions, such as a specific number of days before the renewal date, the system triggers automated alerts to notify users in advance.

The alert mechanism is a central feature of the platform and can be implemented using various communication channels, such as email notifications or push notifications. This ensures that users are consistently informed and can take necessary actions, such as cancelling or modifying subscriptions, before charges are incurred.

The system also includes categorization features, allowing users to organize subscriptions based on type, such as entertainment, productivity, utilities, or education. This enhances clarity and enables better analysis of spending patterns. Additionally, a dashboard interface provides a consolidated view of all active subscriptions, total monthly expenses, and upcoming renewals.

From a technical perspective, the project emphasizes backend development, system design, and scalability. The architecture is designed to handle multiple users and large volumes of data efficiently. Techniques such as database optimization, caching, and asynchronous processing are utilized to ensure high performance and responsiveness.

However, certain features are considered beyond the scope of this project. These include direct integration with payment gateways for automated billing transactions, advanced machine learning-based recommendation systems, and enterprise-level analytics with predictive insights. While these features can significantly enhance the platform, they are reserved for future development phases to maintain the project's feasibility within the given constraints.

1.4 Brief Overview of the Solution

The proposed solution is an automated and scalable subscription management platform designed to streamline the process of tracking, organizing, and managing subscriptions. The platform serves as a centralized system where users can store and monitor all their subscription-related information in a structured and efficient manner.

At its core, the system is built on a robust backend architecture that leverages modern technologies to ensure scalability, reliability, and performance. Subscription data is stored in a structured database, enabling efficient retrieval and processing. The system continuously analyses this data to identify upcoming renewal events and triggers appropriate actions based on predefined rules.

One of the key components of the platform is the notification system, which plays a crucial role in enhancing user awareness. By sending timely alerts before subscription renewals, the system helps users avoid unnecessary charges and make informed decisions. These alerts can be customized based on user preferences, ensuring flexibility and personalization.

The platform follows a modular design approach, where different functionalities are divided into independent components or modules. For example, user management,

subscription tracking, and notification services operate as separate modules that communicate through well-defined interfaces. This modularity not only simplifies development and maintenance but also allows the system to scale efficiently as the number of users increases.

To further enhance performance, the system incorporates techniques such as caching frequently accessed data and using asynchronous processing for time-consuming operations like sending notifications. These optimizations reduce system latency and improve overall responsiveness, ensuring a smooth user experience.

Security and data integrity are also important considerations in the system design. Appropriate measures are implemented to protect user data and ensure that subscription information remains accurate and consistent. This includes validation mechanisms, secure authentication, and controlled access to system resources.

In conclusion, the proposed platform provides a comprehensive solution to the challenges associated with subscription management. By combining automation, scalability, and user-centric design, the system aims to simplify subscription tracking, reduce financial inefficiencies, and enhance user awareness. It represents a practical application of modern software engineering principles and demonstrates the potential of technology in solving real-world problems.

CHAPTER 2: Problem Statement

2.1 Problem Definition

In the modern digital era, the widespread adoption of subscription-based services has significantly transformed how users access and consume products and services. From entertainment platforms and cloud storage solutions to productivity tools and online learning systems, subscriptions have become an integral part of everyday life. While these services offer convenience and flexibility, they also introduce a set of challenges related to effective management and tracking.

The primary problem addressed in this project is the lack of an efficient and centralized system for managing multiple subscriptions. Most users subscribe to numerous services across different platforms, each with its own billing cycle, pricing structure, and renewal policy. Over time, this leads to a fragmented view of active subscriptions, making it difficult for users to maintain control over their financial commitments.

One of the key issues is the absence of visibility. Users often do not have a clear understanding of how many subscriptions they are currently enrolled in or the total amount they are spending on these services. This lack of transparency results in poor financial planning and unnecessary expenditures. In many cases, users continue to pay for services they no longer actively use, simply because they forget to cancel them before the renewal date.

Another significant challenge is the management of renewal schedules. Subscription services typically operate on recurring billing cycles, such as monthly or yearly payments. Without an effective tracking mechanism, users may miss important renewal dates, leading to unintended charges. Existing solutions, such as manual tracking through spreadsheets or setting reminders in calendar applications, are not only time-consuming but also prone to human error.

Additionally, there is a lack of proactive alert systems that can notify users about upcoming renewals. Most platforms do not provide sufficient or timely notifications, and even when they do, these alerts may get overlooked or lost among other communications. As a result, users are unable to make informed decisions regarding continuation or cancellation of services.

From an organizational perspective, small businesses and teams also face difficulties in managing multiple software subscriptions required for their operations. Without a centralized system, tracking expenses, monitoring usage, and ensuring cost efficiency

becomes increasingly complex. This can lead to redundant subscriptions and inefficient allocation of resources.

Furthermore, existing subscription management approaches do not scale well with the increasing number of services. As users adopt more digital tools, the complexity of managing subscriptions grows exponentially. There is a clear need for a system that can handle this scalability while maintaining performance, accuracy, and reliability.

Another aspect of the problem is the lack of integration and automation. Current methods do not leverage modern technologies to automate repetitive tasks such as tracking renewal dates or sending reminders. This not only increases the workload on users but also reduces overall efficiency.

In summary, the problem can be defined as the absence of a centralized, automated, and scalable platform that enables users to effectively track, manage, and receive timely alerts for their subscriptions. Addressing this problem requires a solution that enhances visibility, reduces manual effort, and ensures that users are always informed about their subscription activities.

2.2 Objectives

The primary objective of this project is to design and develop an automated and scalable platform for subscription management that addresses the challenges identified in the problem definition. The system aims to provide users with a centralized and efficient way to manage their subscriptions while ensuring timely notifications and improved financial awareness.

One of the key objectives is to create a centralized repository where users can store and manage all their subscription details in a structured manner. This includes information such as service name, subscription type, billing cycle, cost, and renewal date. By consolidating this information in a single platform, users can gain better visibility and control over their subscriptions.

Another important objective is to implement an automated tracking mechanism for subscription lifecycles. The system should be capable of continuously monitoring subscription data and identifying upcoming renewal events. This eliminates the need for manual tracking and reduces the risk of missed deadlines.

The development of a reliable alert system is also a critical objective. The platform should notify users in advance of upcoming renewals through appropriate channels,

such as email or push notifications. These alerts should be timely, customizable, and consistent, ensuring that users have sufficient time to take necessary actions.

Improving user experience is another major goal of the project. The system should provide an intuitive and user-friendly interface that allows users to easily add, update, and manage their subscriptions. Features such as categorization and dashboard views should enhance usability and provide meaningful insights into subscription usage and expenses.

Scalability and performance are also key objectives of the system design. The platform should be capable of handling multiple users and large volumes of subscription data without compromising on speed or reliability. This involves the use of efficient data storage techniques, optimized queries, and scalable architecture.

Another objective is to incorporate modular design principles in the system architecture. By dividing the system into independent components, such as user management, subscription tracking, and notification services, the platform can achieve better maintainability and flexibility. This approach also allows for easier future enhancements and integration with additional features.

Ensuring data accuracy and consistency is also an essential objective. The system should implement validation mechanisms and error-handling techniques to maintain the integrity of subscription data. This is crucial for generating accurate alerts and reports.

Finally, the project aims to provide a foundation for future improvements and advanced features. While the current implementation focuses on core functionalities, the system should be designed in a way that allows for easy integration of additional capabilities, such as payment gateway integration, analytics, and recommendation systems.

In conclusion, the objectives of this project are centered around creating a robust, automated, and user-centric subscription management platform that enhances efficiency, improves financial awareness, and simplifies the overall process of managing subscriptions.

CHAPTER 3: Problem Statement

3.1 Software Requirement Specifications (SRS)

The Software Requirement Specification (SRS) document provides a comprehensive and structured description of the system's functionality, constraints, and expected behavior. It serves as a critical reference point throughout the software development lifecycle, ensuring that all stakeholders—including developers, designers, and evaluators—have a clear and consistent understanding of the system requirements. By formally documenting the system's features and operational conditions, the SRS minimizes ambiguity and reduces the likelihood of misinterpretation during the implementation phase.

In addition to defining system requirements, the SRS acts as a foundation for subsequent stages such as system design, architecture planning, testing, and deployment. It establishes a clear link between user needs and technical implementation, ensuring that the final product aligns with the intended objectives. A well-defined SRS also facilitates better project management by enabling accurate estimation of resources, time, and effort required for development.

The proposed system, *Automated and Scalable Platform for Subscription Management with Alerts*, is designed to address the growing challenges associated with managing multiple subscription-based services. As users increasingly rely on digital platforms that operate on recurring billing models, the need for a centralized and automated solution has become more prominent. The system aims to simplify subscription management by providing a unified platform where users can store, track, and monitor all their subscriptions in an organized manner.

The SRS for this system outlines both functional and non-functional requirements to ensure clarity, completeness, and consistency. Functional requirements describe the specific operations and features that the system must perform, such as user authentication, subscription tracking, and alert generation. Non-functional requirements, on the other hand, define the quality attributes of the system, including performance, scalability, security, and usability. Together, these requirements provide a holistic view of the system's expected behavior and performance standards.

The system is primarily intended for individual users and small organizations that require a reliable and efficient mechanism for managing subscription services. Individual users often struggle to keep track of multiple subscriptions across different

platforms, leading to missed renewal dates and unnecessary expenses. Similarly, small organizations rely on various software tools and services for their operations, making it essential to monitor subscription costs and usage effectively.

By offering a centralized solution, the system eliminates the need for fragmented and manual tracking methods such as spreadsheets or calendar reminders. It introduces automation into the process, allowing users to receive timely notifications about upcoming renewals and make informed decisions regarding their subscriptions. This not only reduces manual effort but also enhances financial awareness and control.

Furthermore, the system is designed with scalability in mind, ensuring that it can handle an increasing number of users and subscription records without compromising performance. The architecture supports efficient data management and real-time processing, enabling the platform to deliver consistent and reliable results even under high load conditions.

Another important aspect of the system is its focus on user experience. The platform aims to provide an intuitive and user-friendly interface that simplifies interaction and reduces the learning curve for new users. By combining ease of use with powerful backend capabilities, the system ensures that users can effectively manage their subscriptions without requiring advanced technical knowledge.

In conclusion, the SRS document plays a vital role in defining the structure and behavior of the proposed system. It provides a detailed blueprint that guides the development process and ensures that the final implementation meets user expectations. By clearly outlining requirements and constraints, the SRS contributes to the creation of a robust, scalable, and efficient subscription management platform that addresses real-world challenges in a practical and effective manner.

3.1.1 Functional Requirements of the Project

Functional requirements define the core operations and features that the system must perform. These requirements describe how the system should behave in response to user inputs and interactions.

The key functional requirements of the system are as follows:

1. User Registration and Authentication

The system should allow users to create an account using valid credentials such as email and password. It must also support secure login and logout functionalities. Authentication mechanisms should ensure that only authorized users can access their data.

2. Subscription Management

Users should be able to add new subscriptions by providing details such as service name, cost, billing cycle, start date, and renewal date. The system must also allow users to update or delete existing subscription records.

3. Subscription Tracking

The system should continuously track all active subscriptions and maintain accurate records of their status. It must identify upcoming renewal dates and calculate the time remaining for each subscription.

4. Automated Alerts and Notifications

One of the core features of the system is to send automated alerts to users before subscription renewals. The system should allow users to configure notification preferences, such as receiving alerts a specific number of days before the renewal date.

5. Categorization of Subscriptions

Users should be able to categorize subscriptions into predefined or custom categories such as entertainment, productivity, education, and utilities. This helps in better organization and analysis.

6. Dashboard and Reporting

The system should provide a dashboard that displays an overview of all active subscriptions, upcoming renewals, and total expenditure. It should also present data in a structured format for easy understanding.

7. Search and Filter Functionality

Users should be able to search for specific subscriptions and apply filters based on categories, billing cycles, or renewal dates.

8. Data Persistence

The system must ensure that all subscription data is stored securely in a database and remains accessible across sessions.

9. Notification History

The platform should maintain a history of sent notifications, allowing users to review past alerts.

10. Error Handling and Validation

The system should validate user inputs and handle errors gracefully to prevent incorrect data entry and system failures.

3.1.2 Non-Functional Requirements of the Project

Non-functional requirements define the quality attributes and constraints of the system. These requirements ensure that the system performs efficiently, reliably, and securely.

1. Performance Requirements

The system should provide fast response times for user interactions, even when handling a large number of subscriptions. Operations such as adding or retrieving data should be completed within minimal time.

2. Scalability

The platform should be designed to handle an increasing number of users and subscriptions without significant degradation in performance. The architecture should support horizontal scaling if required.

3. Reliability

The system should operate consistently without failures. It must ensure that alerts are delivered accurately and on time.

4. Security

User data must be protected through secure authentication mechanisms and encryption techniques. Unauthorized access to sensitive information should be prevented.

5. Usability

The system should have a simple and intuitive interface that allows users to perform tasks easily without requiring technical expertise.

6. Maintainability

The system should be designed in a modular manner, allowing developers to update or modify components without affecting the entire system.

7. Availability

The platform should be accessible at all times, with minimal downtime. High availability ensures uninterrupted service for users.

8. Compatibility

The system should be compatible with different devices and browsers, ensuring accessibility across platforms.

9. Data Integrity

The system must ensure that stored data remains accurate and consistent throughout its lifecycle.

3.2 Feasibility Study of the Project

The feasibility study evaluates whether the proposed system is practical and viable from different perspectives.

1. Technical Feasibility

The project is technically feasible as it utilizes widely adopted technologies such as backend frameworks, databases, and notification services. The required tools and resources are readily available, and the development process can be carried out using standard software engineering practices.

2. Economic Feasibility

The cost of developing the system is relatively low, as most of the tools and technologies used are open-source. The project does not require expensive hardware or infrastructure, making it economically viable.

3. Operational Feasibility

The system is designed to be user-friendly and requires minimal training for users. Its implementation can significantly improve efficiency in managing subscriptions, making it operationally feasible.

4. Schedule Feasibility

The project can be completed within the given timeframe by following a structured development approach. Proper planning and task allocation ensure timely delivery.

3.3 Tools / Technologies / Platform Used

The development of the system involves the use of modern tools and technologies to ensure performance, scalability, and reliability.

1. Programming Languages

Languages such as Java or JavaScript are used for backend development due to their robustness and wide community support.

2. Backend Framework

Frameworks like Spring Boot or Node.js are used to build scalable and efficient backend services.

3. Database Management System

Relational databases such as MySQL or PostgreSQL are used to store subscription data securely and efficiently.

4. Caching Technologies

Technologies like Redis may be used to improve system performance by reducing database load.

5. Containerization

Docker is used to containerize the application, ensuring consistency across different environments.

6. Message Queues

Tools like Kafka can be used for handling asynchronous tasks such as sending notifications.

7. Version Control System

Git is used for tracking changes in the codebase and enabling collaboration.

8. Development Environment

Integrated Development Environments (IDEs) such as IntelliJ IDEA or Visual Studio Code are used for coding and debugging.

3.4 Architecture Diagrams

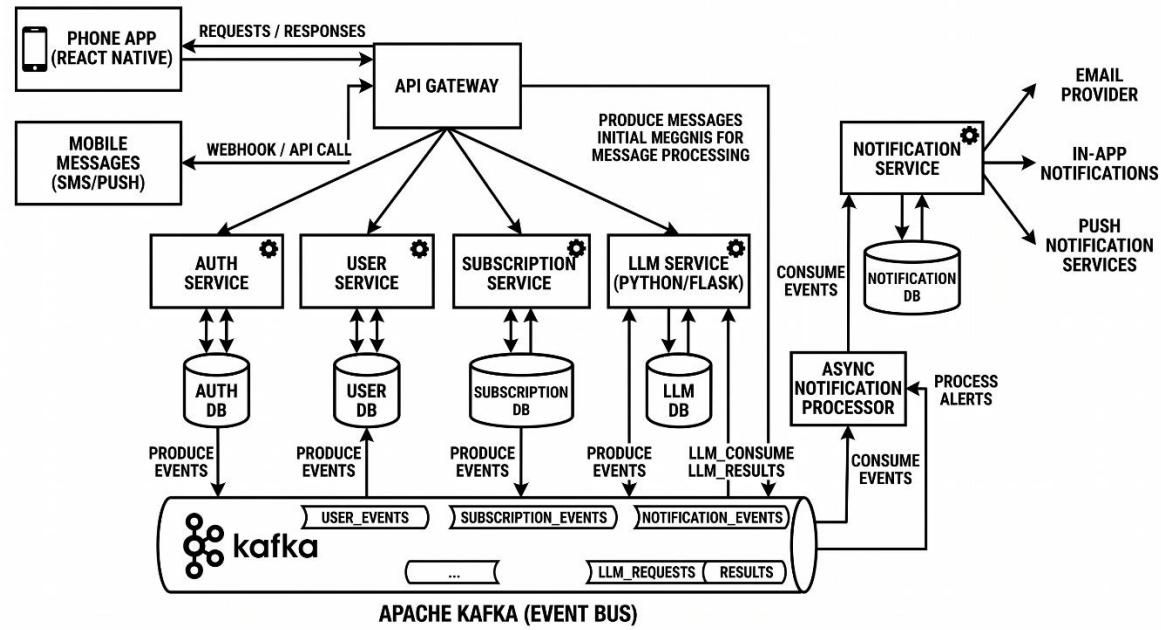

Fig 3.4: Architecture Diagram

CHAPTER 4: Design and Architecture

4.1 Structure Chart / Work Breakdown Structure

The Structure Chart and Work Breakdown Structure (WBS) provide a hierarchical representation of the system's components and the overall development process. These tools help in organizing the system into manageable units, ensuring clarity in design and efficient allocation of responsibilities.

The Work Breakdown Structure divides the entire project into smaller, well-defined tasks and sub-tasks. It begins with the highest-level objective, which is the development of the *Automated and Scalable Platform for Subscription Management with Alerts*, and breaks it down into major functional areas such as requirement analysis, system design, development, testing, and deployment.

Each of these phases is further decomposed into specific activities. For example, the development phase includes backend development, database design, API integration, and notification system implementation. This hierarchical decomposition ensures that each task is clearly defined and can be executed systematically.

The Structure Chart, on the other hand, represents the modular organization of the system. It illustrates how different modules interact with each other and how control flows within the system. The top-level module represents the main system, which is further divided into submodules such as user management, subscription management, and notification services.

This structured approach offers several advantages. It improves maintainability by isolating functionalities into independent modules, facilitates parallel development by allowing different components to be developed simultaneously, and enhances scalability by enabling the addition of new modules without affecting existing ones.

4.2 Explanation of Modules

The system is designed using a modular architecture, where each module is responsible for a specific functionality. This approach ensures separation of concerns, making the system easier to develop, test, and maintain.

1. User Management Module

This module handles all user-related operations, including registration, authentication, and profile management. It ensures secure access to the system by validating user

credentials and managing sessions. The module also supports updating user information and maintaining user-specific preferences.

2. Subscription Management Module

This is the core module of the system. It allows users to add, update, delete, and view subscription details. Each subscription record includes attributes such as service name, cost, billing cycle, start date, and renewal date. The module ensures accurate tracking of all active subscriptions.

3. Notification Module

The notification module is responsible for sending alerts to users regarding upcoming subscription renewals. It processes subscription data, identifies renewal events, and triggers notifications based on predefined conditions. This module may use asynchronous processing to ensure timely delivery without affecting system performance.

4. Dashboard Module

This module provides a visual representation of subscription data. It displays key information such as total expenses, upcoming renewals, and categorized subscriptions. The dashboard enhances user experience by presenting data in an organized and easily understandable format.

5. Database Module

The database module manages data storage and retrieval. It ensures that all subscription and user data is stored securely and efficiently. Proper indexing and query optimization techniques are used to improve performance.

6. API Layer

The API layer acts as an interface between the frontend and backend components. It handles requests from the user interface and communicates with the backend services to process data. This layer ensures smooth interaction between different parts of the system.

4.3 Flow Chart / Activity Diagram

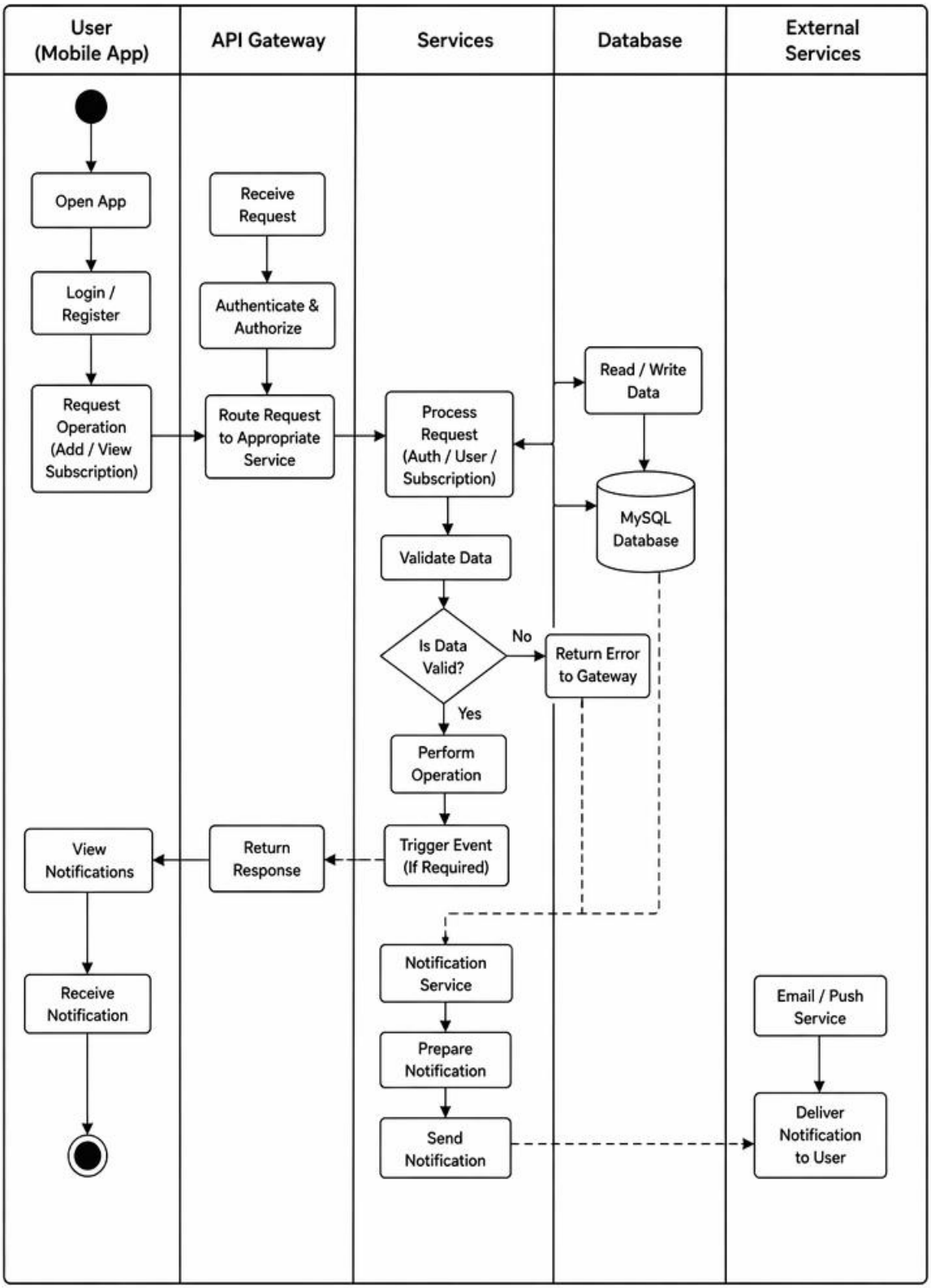

Fig 4.3.1: Flow Chart

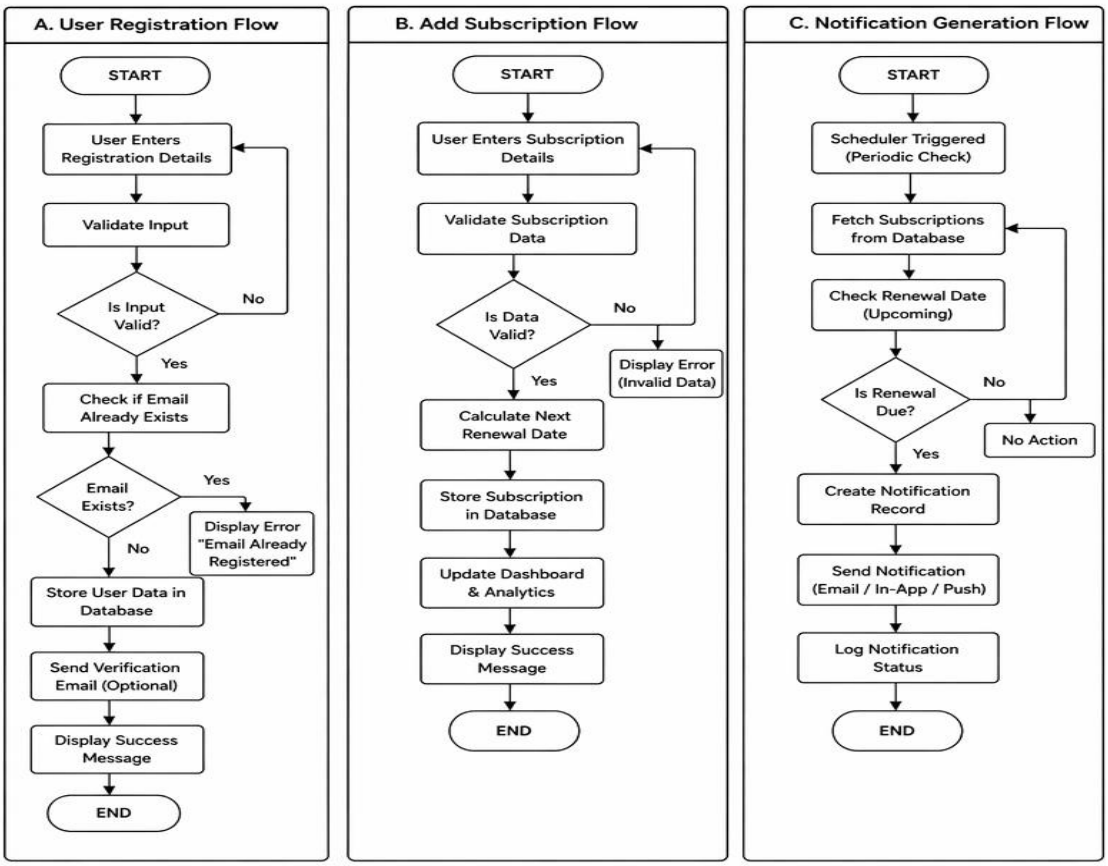

Fig 4.3.2: Activity Diagram

4.4 ER Diagram

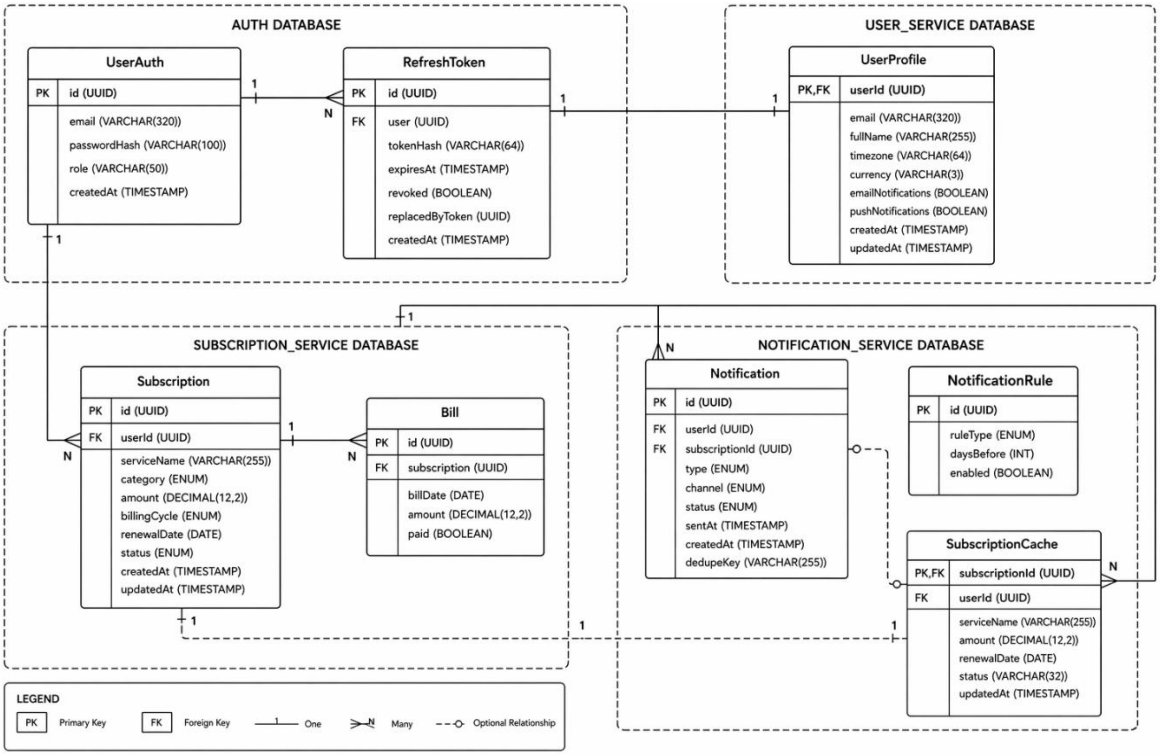

Fig 4.4: ER Diagram

CHAPTER 5: Implementation

5.1 Screenshots

The implementation phase focuses on translating the system design into a working application. This includes developing the backend services, integrating the database, and creating a user interface for interaction. The following section presents key screenshots of the system along with their explanations to demonstrate the functionality of the platform.

1. User Registration (Create Account Page)

The Create Account page allows new users to register on the platform by entering essential details such as full name, email address, and password. The system implements input validation to ensure that all required fields are filled correctly and that the email format is valid. Password constraints such as minimum length and strength may also be enforced to enhance security.

The login page enables registered users to securely access their accounts by entering valid credentials. The system authenticates the user using secure mechanisms and generates a session or token upon successful login.

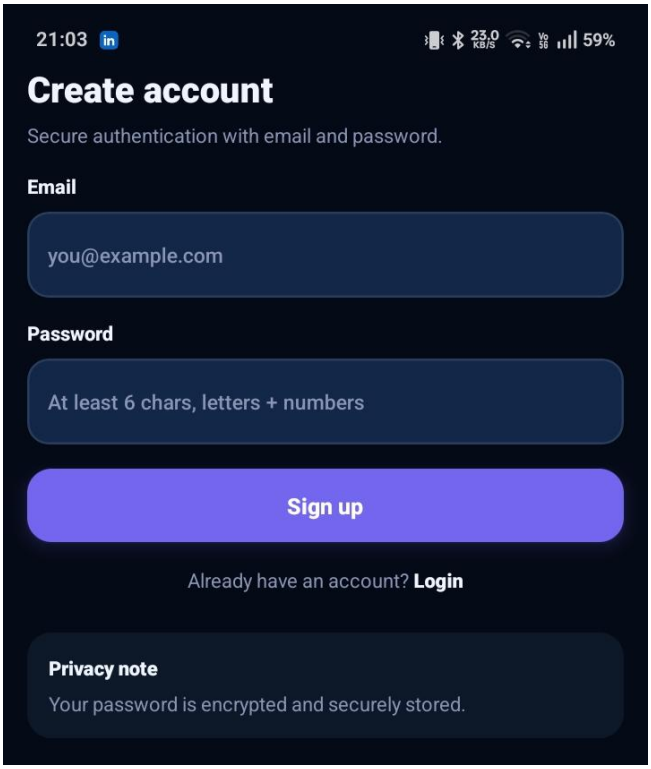

Fig 5.1: User Registration

2. Dashboard Interface

The dashboard serves as the central hub of the application, providing a comprehensive overview of the user's subscription data. It displays key metrics such as total active subscriptions, upcoming renewals, and overall spending.

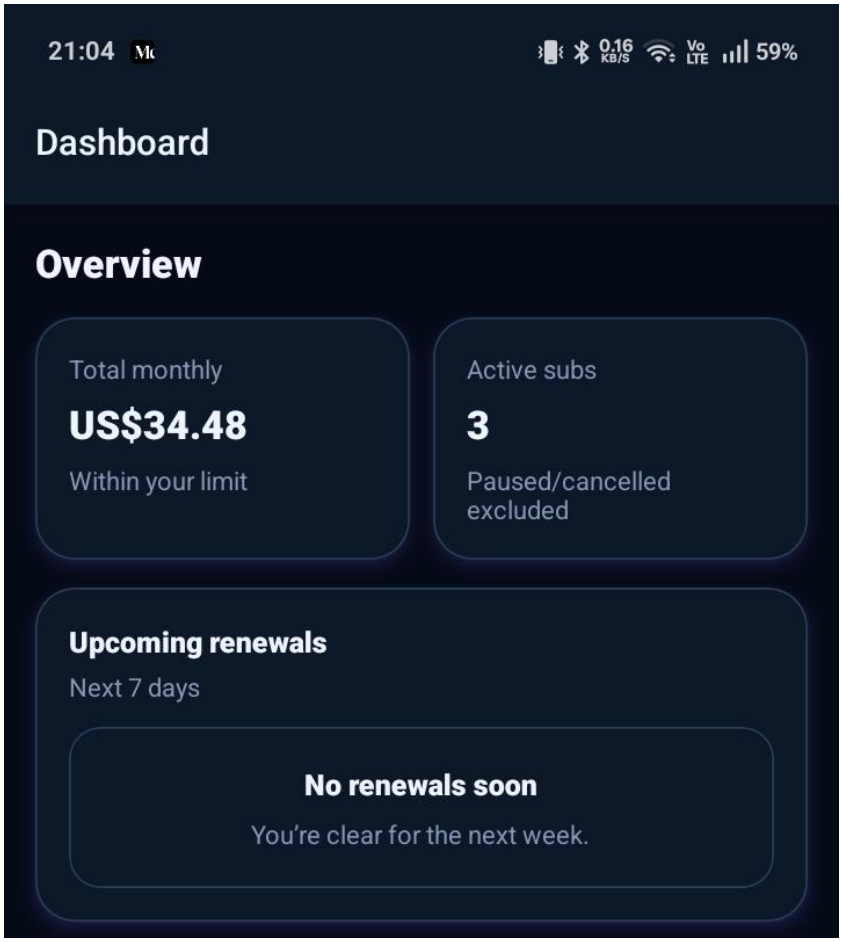

Fig 5.2: Dashboard

The dashboard is designed to present information in a visually structured and easy-to-understand format. It allows users to quickly analyse their subscription status without navigating through multiple pages.

3. Subscription List Page

The subscription page displays all active subscriptions in a structured list format. Each entry includes important details such as service name, cost, billing cycle, and renewal date.

Users can easily browse through their subscriptions and perform actions such as

viewing details, editing, or deleting records. This screen ensures efficient management of subscription data.

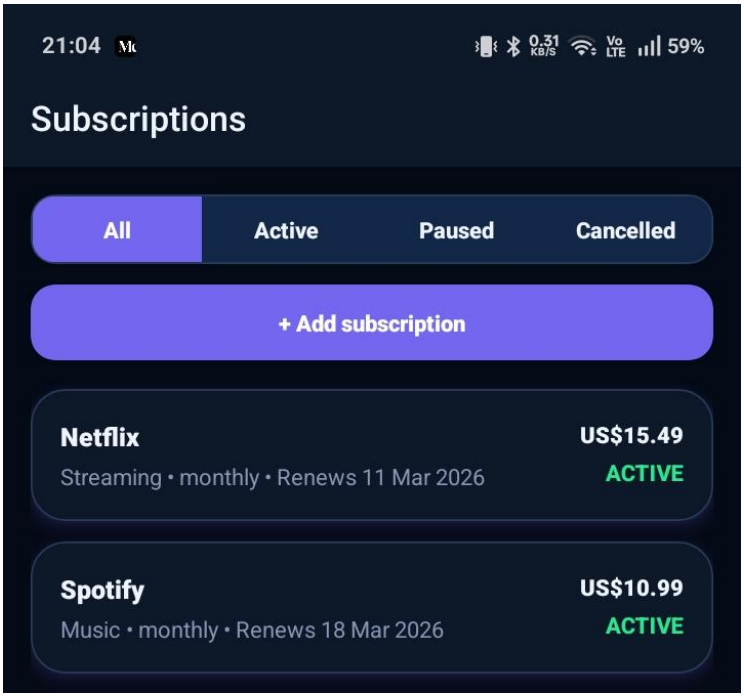

Fig 5.3: Subscription

4. Add Subscription Page

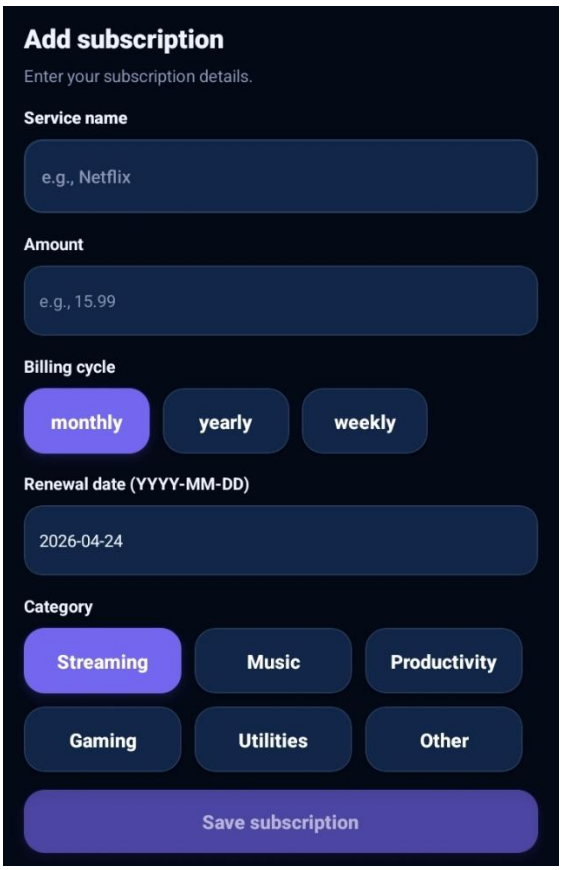

Fig 5.4: Add Subscription

This interface is designed for simplicity and efficiency, enabling users to quickly add new subscriptions without confusion or complexity.

5. Subscription Details Page

The details page provides an in-depth view of a selected subscription. It displays all associated information, including billing history, renewal dates, and status.

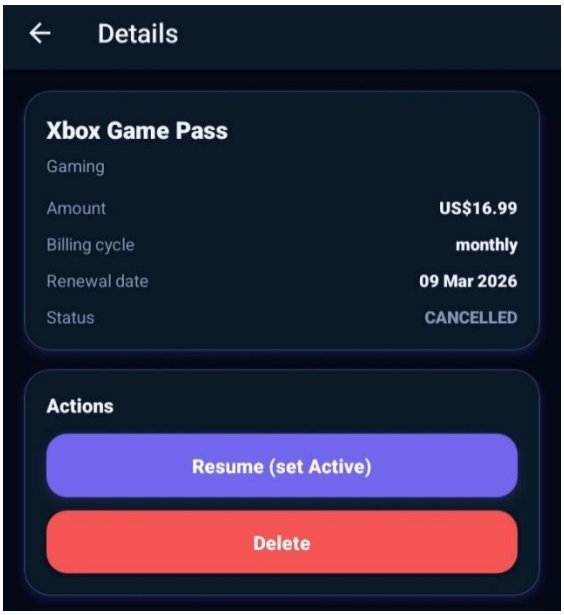

Fig 5.5: Subscription Details

6. Analytics Page

The analytics page presents visual insights into subscription usage and spending patterns. It may include charts or graphs showing monthly expenses, category-wise distribution, and trends over time.

Fig 5.6: Analytic Page

7. Notification Page

The notification page displays alerts related to subscription renewals and other important events. Notifications are generated automatically based on predefined rules.

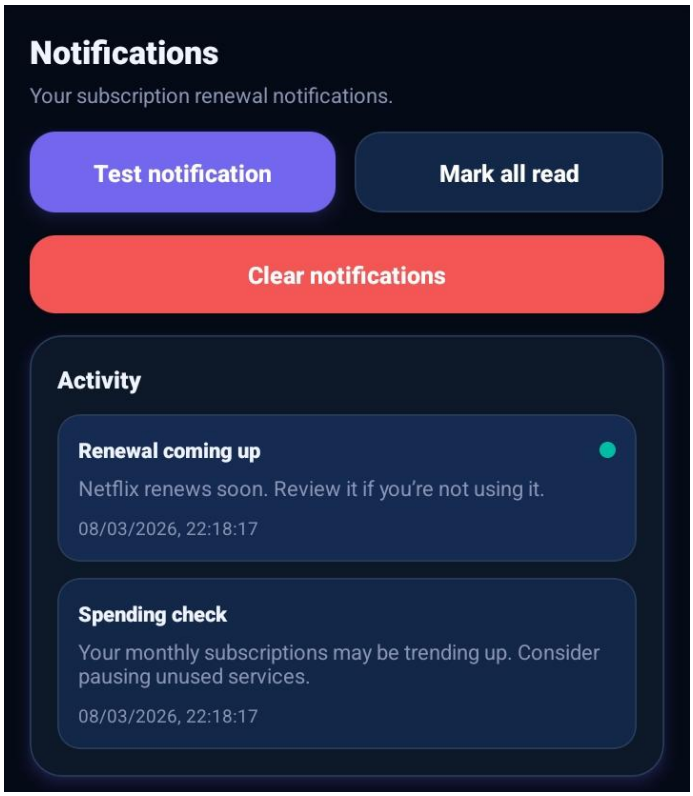

Fig 5.7: Notification

8. Help / Support Page

The help page provides assistance to users by offering guidance on how to use the platform. It may include FAQs, instructions, or contact support options.

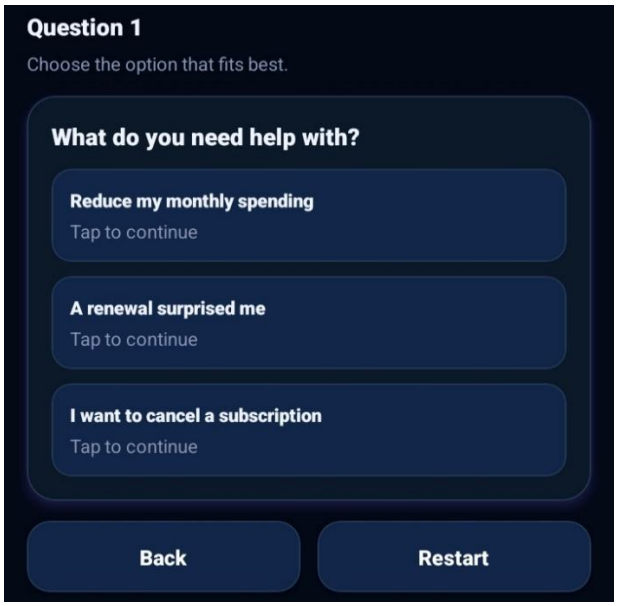

Fig 5.8: Help/Support Page

9. Profile Page

The profile page allows users to manage their personal information and preferences. Users can update details such as name, email, password, and notification settings.

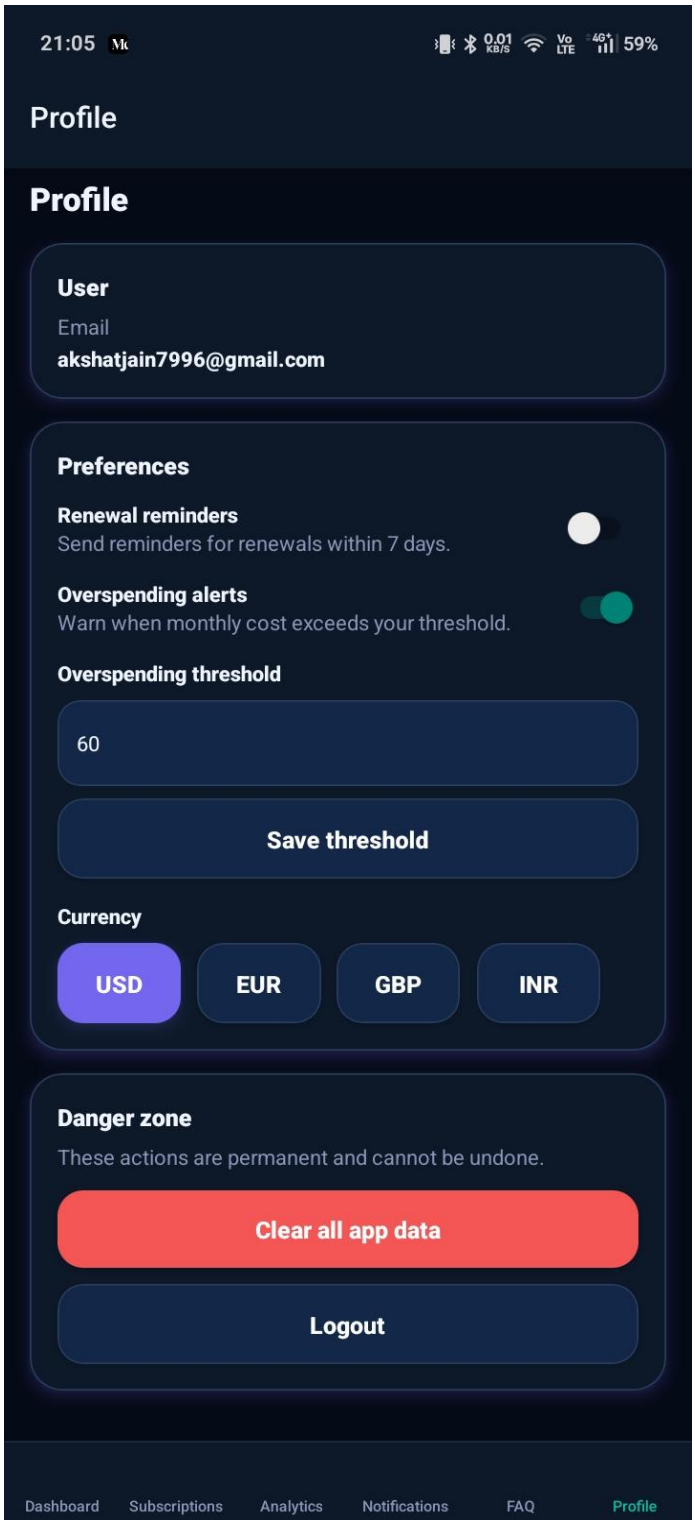

Fig 5.9: Profile Page

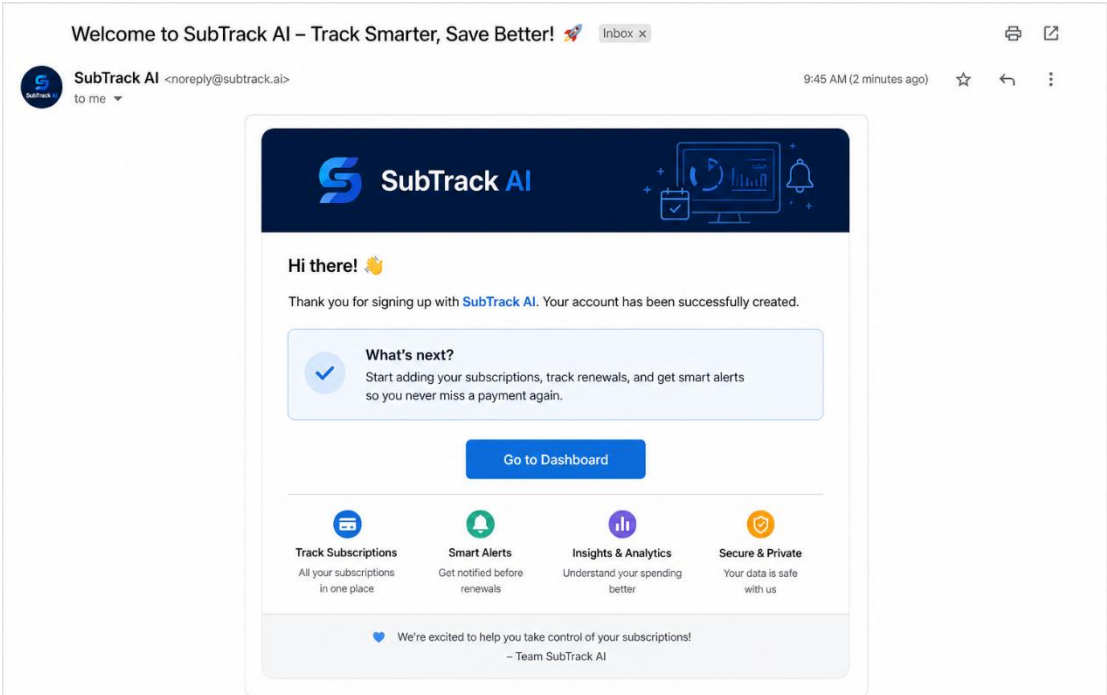

Fig 5.10: Onboarding Mail

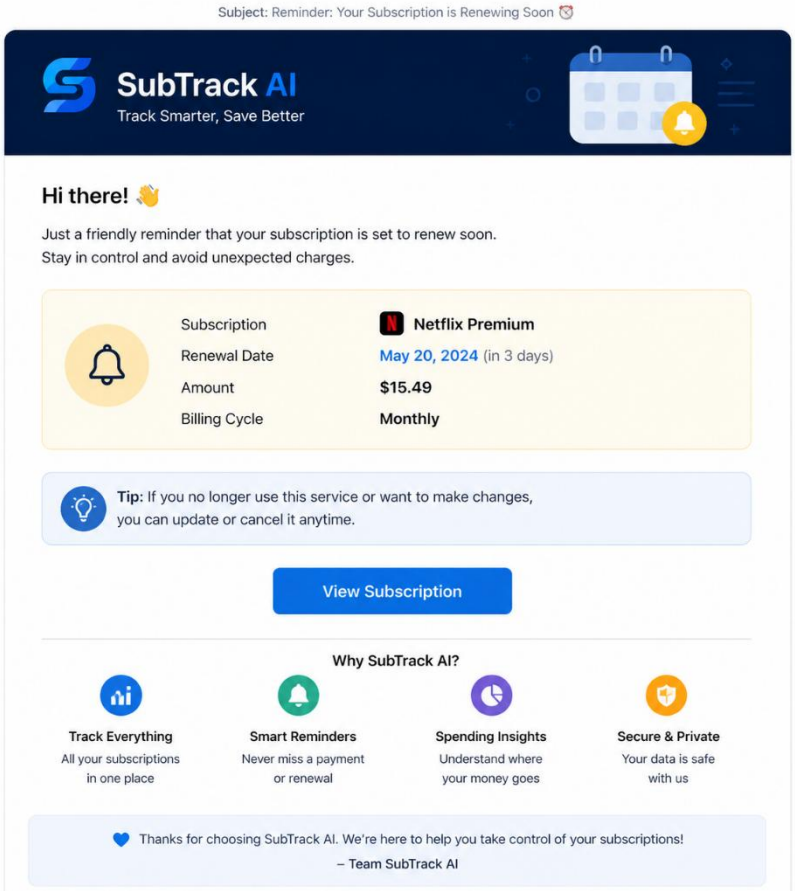

Fig 5.11: Reminder Mail

5.2 Source Code of Some Modules

1. API Gateway Service (Routing Configuration)

```
@Configuration
public class GatewayConfig {

    @Bean
    public RouteLocator customRoutes(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("auth-service", r -> r.path("/api/auth/**")
                .uri("http://localhost:8081"))
            .route("user-service", r -> r.path("/api/users/**")
                .uri("http://localhost:8082"))
            .route("subscription-service", r -> r.path("/api/subscriptions/**")
                .uri("http://localhost:8083"))
            .route("notification-service", r -> r.path("/api/notifications/**")
                .uri("http://localhost:8084"))
            .build();
    }
}
```

Explanation:

This module defines routing rules in the API Gateway. It forwards incoming client requests to the appropriate microservice based on URL patterns, enabling centralized request handling.

2. Auth Service (JWT Authentication Controller)

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    @Autowired
    private JwtService jwtService;

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest request) {
        if (authenticate(request)) {
            String token = jwtService.generateToken(request.getEmail());
            return ResponseEntity.ok(token);
        }
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
credentials");
    }
    private boolean authenticate(LoginRequest request) {
        return "test@example.com".equals(request.getEmail() &&
"password".equals(request.getPassword());
    }
}
```

Explanation:

This controller handles user login and generates JWT tokens for authenticated users, ensuring secure access across services.

3. User Service (Profile Management)

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public ResponseEntity<UserProfile> getUser(@PathVariable String id) {
        UserProfile user = new UserProfile(id, "Akshat Jain", "akshat@email.com");
        return ResponseEntity.ok(user);
    }
}
```

Explanation:

This module retrieves user profile information. It demonstrates how user-related data is managed independently in the user service.

4. Subscription Service (CRUD Operation)

```
@RestController
@RequestMapping("/api/subscriptions")
public class SubscriptionController {

    @PostMapping("/add")
    public ResponseEntity<String> addSubscription(@RequestBody Subscription sub)
{
        return ResponseEntity.ok("Subscription added: " + sub.getServiceName());
    }

    @GetMapping("/{userId}")
    public List<Subscription> getSubscriptions(@PathVariable String userId) {
        return List.of(new Subscription("Netflix", 499));
    }
}
```

Explanation:

This service handles subscription-related operations such as adding and retrieving subscriptions for a user.

5. Notification Service (Alert Logic)

```
@Service
public class NotificationService {

    public void sendNotification(String message) {
        System.out.println("Notification Sent: " + message);
    }

    public void checkRenewals(LocalDate renewalDate) {
        if (renewalDate.isBefore(LocalDate.now().plusDays(3))) {
            sendNotification("Subscription renewal coming soon!");
        }
    }
}
```

Explanation:

This module processes subscription data and triggers alerts for upcoming renewals.

6. LLM Service (Python - Message Processing)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/process', methods=['POST'])
def process_message():
    data = request.json
    message = data.get("message")

    # Simple extraction logic (mock)
    if "netflix" in message.lower():
        return jsonify({
            "service": "Netflix",
            "amount": 499
        })

    return jsonify({"status": "unknown"})

if __name__ == '__main__':
    app.run(port=5000)
```

Explanation:

This service processes user messages and extracts subscription-related information, enabling intelligent automation.

7. Mobile App (React Native - Dashboard Screen)

```
import React from 'react';
import { View, Text } from 'react-native';

const Dashboard = () => {
  return (
    <View>
      <Text>Total Subscriptions: 5</Text>
      <Text>Monthly Spend: ₹1500</Text>
    </View>
  );
};
export default Dashboard;
```

Explanation:

This code represents a simple mobile dashboard screen displaying key subscription metrics to the user.

Chapter 6: Testing

Testing is a crucial phase in the software development lifecycle that ensures the system functions correctly, meets user requirements, and is free from critical defects. The primary objective of testing is to identify and fix errors, validate system behavior, and ensure that the application performs reliably under different conditions.

For the *Automated and Scalable Platform for Subscription Management with Alerts*, various testing techniques were applied to verify the correctness, performance, and usability of the system. The testing process was carried out systematically, covering different levels such as unit testing, integration testing, and system testing.

6.1 Objectives of Testing

The main objectives of testing in this project are:

- To ensure that all functionalities work as expected
- To identify and fix bugs or inconsistencies
- To validate system performance and reliability
- To ensure proper handling of user inputs and errors
- To verify that notifications are triggered correctly
- To confirm that the system meets all specified requirements

6.2 Types of Testing Performed

1. Unit Testing

Unit testing involves testing individual components or modules of the system independently. Each function or method is tested to ensure it performs the intended operation correctly. For example, functions related to adding subscriptions or validating user input are tested in isolation.

2. Integration Testing

Integration testing ensures that different modules of the system work together seamlessly. For instance, the interaction between the subscription module and the notification module is tested to verify that alerts are triggered correctly when renewal dates approach.

3. System Testing

System testing evaluates the complete system as a whole. It ensures that all functionalities are working together as expected and that the system meets the specified requirements.

4. User Acceptance Testing (UAT)

User Acceptance Testing is performed to verify that the system meets user expectations. It involves testing the application from an end-user perspective to ensure usability and functionality.

5. Performance Testing

Performance testing checks how the system behaves under different workloads. It ensures that the system remains responsive even when handling multiple users or large datasets.

6. Security Testing

Security testing ensures that user data is protected and unauthorized access is prevented. It includes testing authentication mechanisms and data validation processes.

6.3 Test Cases

The following table represents sample test cases used to validate system functionality:

Test ID	Description	Input	Expected Output	Actual Output	Status
TC01	User Registration	Valid user details	User registered successfully	Same as expected	Pass
TC02	User Login	Correct credentials	Login successful	Same as expected	Pass
TC03	User Login	Incorrect credentials	Error message	Same as expected	Pass
TC04	Add Subscription	Valid subscription data	Subscription added	Same as expected	Pass
TC05	Add Subscription	Missing fields	Validation error	Same as expected	Pass
TC06	Delete Subscription	Valid ID	Subscription deleted	Same as expected	Pass
TC07	Notification Trigger	Renewal date near	Alert generated	Same as expected	Pass
TC08	Dashboard Load	Multiple subscriptions	Data displayed correctly	Same as expected	Pass

Table 6.3: Test Case

6.4 Testing Tools Used

The testing process utilized various tools and techniques, including:

- Manual testing for functional validation
- Postman for API testing
- Browser-based testing for UI validation
- Logging tools for debugging and error tracking

6.5 Results and Analysis

The testing phase confirmed that the system meets the specified functional and non-functional requirements. All major functionalities, including user authentication, subscription management, and notification alerts, were tested and validated successfully.

The system demonstrated stable performance under normal operating conditions and handled user inputs effectively. Minor issues identified during testing were resolved, resulting in a reliable and efficient application.

6.6 Conclusion of Testing

In conclusion, the testing process ensured that the system is robust, reliable, and ready for deployment. The application successfully meets user requirements and performs efficiently under various scenarios. Proper testing has significantly improved system quality and reduced the likelihood of errors in real-world usage.

Chapter 7: Summary and Conclusion

7.1 Summary

The project titled *"Automated and Scalable Platform for Subscription Management with Alerts"* was developed with the primary objective of simplifying and optimizing the process of managing multiple subscription-based services. In today's digital environment, where users increasingly rely on numerous platforms operating on recurring billing models, managing subscriptions efficiently has become a significant challenge. This project addresses that challenge by providing a centralized and automated solution.

Throughout the development of this system, various phases of the software development lifecycle were carefully followed, including requirement analysis, system design, implementation, and testing. Each phase played a crucial role in ensuring that the final product meets both functional and non-functional requirements.

In the analysis phase, the problem of inefficient subscription management was studied in detail, and a comprehensive Software Requirement Specification (SRS) was prepared. This helped in clearly defining the scope, functionalities, and constraints of the system. The design phase focused on creating a modular and scalable architecture, supported by diagrams such as use case diagrams, data flow diagrams, and class diagrams.

During the implementation phase, the system was developed using modern technologies and tools, ensuring efficiency and scalability. Key features such as user authentication, subscription tracking, and automated notifications were successfully implemented. The system allows users to add, update, and manage their subscriptions while providing timely alerts for upcoming renewals.

The testing phase ensured that all components of the system functioned correctly and reliably. Different types of testing, including unit testing, integration testing, and system testing, were performed to validate system behavior. Test cases were executed to verify the accuracy of functionalities and identify potential issues, which were resolved to improve system performance.

Overall, the project successfully achieved its objectives by providing a functional and user-friendly platform for managing subscriptions. It demonstrates the practical application of software engineering principles and highlights the importance of automation in solving real-world problems.

7.2 Conclusion

In conclusion, the *Automated and Scalable Platform for Subscription Management with Alerts* effectively addresses the challenges associated with tracking and managing multiple subscriptions. By offering a centralized system with automated alert functionality, the platform enhances user awareness and reduces the likelihood of missed renewals and unnecessary expenses.

The system provides significant benefits, including improved visibility of active subscriptions, better financial control, and reduced manual effort. Its modular architecture ensures scalability and maintainability, making it suitable for future enhancements and integration with additional features.

One of the key strengths of the system is its ability to automate repetitive tasks, such as monitoring renewal dates and sending notifications. This not only improves efficiency but also enhances the overall user experience. The inclusion of a dashboard and categorization features further adds value by providing meaningful insights into subscription usage.

From a technical perspective, the project demonstrates the successful implementation of modern development practices, including structured design, modular programming, and efficient data management. It also highlights the importance of testing in ensuring system reliability and performance.

Despite its strengths, the system has certain limitations, such as the lack of direct integration with payment gateways and advanced analytics features. However, these limitations provide opportunities for future improvements and expansion.

In summary, the project fulfils its intended purpose and provides a practical solution to a common problem faced by many users. It showcases how technology can be leveraged to enhance convenience, improve efficiency, and support better decision-making in everyday life.

Chapter 8: Limitations and Future Work

8.1 Limitations of the Project

While the *Automated and Scalable Platform for Subscription Management with Alerts* successfully achieves its primary objectives, there are certain limitations that need to be acknowledged. Identifying these limitations is important for understanding the current scope of the system and for guiding future improvements.

One of the primary limitations of the system is the lack of direct integration with payment gateways or banking systems. Currently, users are required to manually enter subscription details such as cost, billing cycle, and renewal dates. The absence of automatic data synchronization with financial accounts or service providers may lead to inaccuracies if users fail to update their information regularly.

Another limitation is the dependency on predefined notification mechanisms. Although the system provides timely alerts based on renewal dates, it does not yet support advanced notification customization such as multi-channel alerts (SMS, mobile push notifications) or adaptive reminders based on user behavior. This restricts the flexibility of the alert system.

The system also has limited analytical capabilities. While it provides basic insights such as total subscription costs and upcoming renewals, it does not include advanced analytics features such as spending trends, predictive analysis, or personalized recommendations. These features could significantly enhance user decision-making.

Scalability, although considered during system design, may still face challenges in real-world large-scale deployment. The current implementation is suitable for individual users and small organizations, but it may require further optimization and infrastructure support to handle enterprise-level workloads and high concurrency scenarios.

Another limitation lies in the user interface, which may be relatively basic in the current version. While the system focuses primarily on backend functionality and performance, the user interface can be further improved to provide a more engaging and visually appealing experience.

Security is also an area that can be enhanced. Although basic authentication and validation mechanisms are implemented, the system does not yet include advanced security features such as multi-factor authentication, role-based access control, or encryption of sensitive data at all levels.

Additionally, the system does not currently support integration with third-party services such as subscription providers, calendar applications, or financial management tools. This limits its ability to function as a fully interconnected ecosystem.

8.2 Future Work

The limitations identified in the current system open up several opportunities for future enhancements and improvements. Expanding the system's capabilities can significantly increase its usability, efficiency, and overall impact.

One of the most important areas for future work is the integration with payment gateways and banking APIs. This would enable automatic detection and tracking of subscriptions, eliminating the need for manual data entry. Such integration would improve accuracy and provide real-time updates on subscription activities.

Enhancing the notification system is another key area for development. Future versions of the platform can include multi-channel notifications, such as SMS, mobile push notifications, and in-app alerts. Additionally, intelligent notification systems can be developed to prioritize alerts based on user preferences and behavior patterns.

The inclusion of advanced analytics and reporting features would greatly enhance the platform's functionality. Features such as monthly spending trends, category-wise expense analysis, and predictive insights can help users make informed financial decisions. Machine learning techniques can also be explored to provide personalized recommendations for subscription management.

Improving scalability and performance is another important direction for future work. Implementing distributed systems, load balancing, and cloud-based deployment can enable the platform to support a larger user base and handle high traffic efficiently.

The user interface can also be enhanced to provide a more interactive and visually appealing experience. Modern UI/UX design principles, along with responsive design techniques, can be used to improve accessibility across different devices, including mobile platforms.

Security enhancements are essential for future development. Features such as multi-factor authentication, secure data encryption, and advanced access control mechanisms can be implemented to ensure better protection of user data.

Integration with third-party services is another promising area. Connecting the platform with calendar applications can allow automatic scheduling of reminders, while

integration with financial tools can provide a more comprehensive view of user expenses.

Finally, the system can be extended to support enterprise-level features, such as team-based subscription management, role-based access control, and detailed reporting for organizations. This would expand the platform's applicability beyond individual users to larger businesses and institutions.

Conclusion of Chapter

In conclusion, while the current system provides a solid foundation for managing subscriptions efficiently, there is significant scope for future enhancements. Addressing the identified limitations and implementing advanced features can transform the platform into a comprehensive and intelligent subscription management solution.

Bibliography

[1] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson Education, 2015.

[2] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. New York, NY, USA: McGraw-Hill Education, 2015.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 6th ed. New York, NY, USA: McGraw-Hill Education, 2011.

[4] M. Fowler, *Patterns of Enterprise Application Architecture*. Boston, MA, USA: Addison-Wesley, 2003.

[5] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2004.

[6] Oracle Corporation, "Java Platform, Standard Edition Documentation." [Online]. Available: https://docs.oracle.com/javase/

[7] Pivotal Software, Inc., "Spring Boot Reference Documentation." [Online]. Available: https://spring.io/projects/spring-boot

[8] Docker Inc., "Docker Documentation: Containerization Platform." [Online]. Available: https://docs.docker.com/

[9] Apache Software Foundation, "Apache Kafka Documentation." [Online]. Available: https://kafka.apache.org/documentation/